

Bài 7

CHƯƠNG VI. PHÂN TÍCH THUẬT TOÁN

I. Khái niệm

Mỗi thuật toán phải đảm bảo các tính chất:

- Các bước phải được xác định rõ ràng, rành mạch
- Tổng quát, phải xét được mọi khả năng có thể xảy ra
- Dễ cài đặt chương trình máy tính;
- Hữu hạn : thời gian thực hiện thuật toán chấp nhận được.

Trong chương trình, ta đã học các phương pháp trình bày thuật toán, các dạng thuật toán cơ bản. Một bài toán có thể có nhiều thuật toán giải khác nhau, việc chọn thuật toán nào để viết chương trình cho máy tính đã được các chuyên gia tin học đề cập đến từ lâu, và một lĩnh vực mới đã hình thành : nghiên cứu và phân tích độ hiệu quả của thuật toán.

Những vấn đề cần quan tâm khi xây dựng thuật toán :

- + Cần hiểu rõ bài toán
- + Cần phân tích ở dạng gạch đầu dòng mọi khả năng, mọi trường hợp có thể xảy ra, viết rõ ra được những phép toán trong từng khả năng, từng trường hợp ;
- + Dùng các ký pháp vẽ sơ đồ thuật toán (các hình elip, chữ nhật, bình hành, hình thoi,...)
- + Xác định hiệu quả của thuật toán.

Việc phân tích các thuật toán giải một bài toán cụ thể cho phép ta đánh giá thuật toán nào sẽ có hiệu quả hơn để dẫn tới việc lập trình.

II. Độ hiệu quả và độ phức tạp của thuật toán

2.1. Các loại bài toán.

Độ phức tạp của thuật toán thường phụ thuộc vào từng loại bài toán cần giải quyết. Có bài toán có khối lượng xử lý thông tin rất lớn (như bài toán thống kê dân số, ...), có bài toán có ít thông tin ban đầu nhưng lại rất phức tạp về mặt toán học, logic tính toán. Ta gọi chung các bài toán cần xử lý trên máy tính là công việc tính toán.

Ta có thể phân chia các công việc tính toán cần xử lý trên máy tính ra các lĩnh vực sau :

- Quản lý doanh nghiệp, quản lý nhà nước - chính phủ và quản lý ở các cấp bộ ngành thuộc chính phủ, quản lý nhà nước ở cấp tỉnh và các sở thuộc tỉnh, quản lý tài chính- kế toán- ngân hàng, quản lý pháp. quản lý tòa án, quản lý giáo dục đào tạo...
- Khoa học kỹ thuật thuần túy.
- Lĩnh vực khoa học cơ bản.
- Trong cơ khí, kiến trúc, xây dựng nhà, xây dựng cầu, đường, thủy lợi, giao thông, nông nghiệp v.v.
- Trong mạng : dịch vụ tra cứu thông tin trên mạng, truyền điện tử, thương mại điện tử v.v. .
- Lĩnh vực trò chơi, dạy học, du lịch, dịch vụ công cộng v.v..

2.2. Đánh giá độ hiệu quả, độ phức tạp của thuật toán

2.2.1. Ví dụ dẫn nhập việc tính thời gian thực hiện thuật toán.

Để đánh giá thời gian thực hiện một thuật toán, phải đếm được số thao tác cơ bản mà thuật toán đó thực hiện, mỗi thao tác cơ bản thực hiện bao nhiêu lần, trong mỗi lần thực hiện bao nhiêu phép toán...

Những thao tác cơ bản có thể là phép đọc nhập dữ liệu; hiện dữ liệu; phép gán; cộng, trừ, nhân, chia, so sánh ...

Thời gian thực hiện một thao tác cơ bản được tính là 1 đơn vị thời gian,

Nếu một thao tác bao gồm nhiều thao tác cơ bản thì thời gian thực hiện nó là số lượng các thao tác cơ bản;

Ví dụ :

Trong thao tác `if(điều kiện) S1 else S2`; thời gian thực hiện là cận trên của thời gian thực hiện lệnh S1, S2. Thời gian để thực hiện S1 là t_1 , thời gian để thực hiện S2 là t_2 .

Khi đó thời gian để thực hiện `if(điều kiện) S1 else S2`; là $\max(t_1, t_2)$ - là cận trên của (t_1, t_2)

Cụ thể : nếu $t_1 = 8$, $t_2 = 10$ thì $\max(t_1, t_2) = 10$

Xét bài toán : Nhập vào từ bàn phím một dãy $a[]$ có n số, đưa ra số lớn nhất (max).

Thuật toán có dạng :

1. Nhập n , nhập $a[1]$; gán $\max = a[1]$; (có 3 thao tác cơ bản)
2. `for(i=2; i<=n; i++)`
 { nhập $a[i]$;
 `if(max < a[i]) {max = a[i];}`
 } – lặp $(n-1)$ lần, trong mỗi lần lặp thực hiện tối đa 3 thao tác cơ bản
3. Hiện $\max$. 1 thao tác cơ bản

Tính xem có bao nhiêu phép tính cơ bản phải thực hiện :

- nhập n
- n lần nhập $a[i]$
- $(n-1)$ lần thực hiện lệnh `if` , trong đó mỗi lần thực hiện 1 phép so sánh ($\max < a[i]$) và tối đa 1 phép gán $\max = a[i]$.
- Hiện $\max$

Như vậy, tổng số tối đa phép toán cơ bản phải thực hiện là n lần lặp, mỗi lần thực hiện 3 phép toán, tổng cộng là $3n$.

2.2.2. Khái niệm độ phức tạp của thuật toán

Một thuật toán có thể đ-ợc cài đặt trong máy theo các ngôn ngữ lập trình khác nhau, mỗi cách đòi hỏi phải dùng số l-ợng bộ nhớ và thời gian tính toán nhất định. Cách nào mà sử dụng ít bộ nhớ nhất và thời gian tính toán ít nhất thì nó là hiệu quả nhất. Ngày nay bộ nhớ không còn là vấn đề nên ng-ời ta chỉ quan tâm đến thời gian tính toán. Yếu tố quan trọng nhất ảnh h-ởng đến thời gian tính toán là khối l-ợng thông tin đầu vào.

Ký hiệu T là thời gian thực hiện của thuật toán, khi đó $T(n)$ là một hàm số phụ thuộc vào số lượng đầu vào $n : T(n)$.

Khái niệm **độ phức tạp của thuật toán** được hiểu theo ý nghĩa là độ lớn của $T(n)$, được ký hiệu bằng chữ O (đọc là ô lớn) như sau : giả sử n là số nguyên dương, $T(n)$ và $f(n)$ là các hàm thực không âm, khi đó $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$.

Theo định nghĩa trên, hàm $f(n)$ là **cận trên** của $T(n)$.

Một thuật toán có thời gian thực hiện $T(n) = O(f(n))$ - ta nói rằng nó có thời gian thực hiện cấp $f(n)$. Hàm $O(f(n))$ đọc là O-lớn của $f(n)$, tiếng Anh là Big-O

Viết lại định nghĩa : $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$

Ví dụ. với bài toán nhập vào từ bàn phím một dãy $a[]$ có n số, đưa ra số lớn nhất (max), thời gian thực hiện $T(n)=3n$, theo định nghĩa ta có thể viết :

$T(n)=O(n)$ với $c=3, n_0>2, T(n) \leq 3n$ với mọi $n \geq 2$

Trở lại phần trên, mỗi thao tác cơ bản sẽ có thời gian thực hiện với độ phức tạp là $O(1)$;

Thuật toán trong ví dụ 1 có thời gian thực hiện với độ phức tạp là $O(n)$.

Giải thích ý nghĩa của hàm O lớn (Big O).

Khi nghiên cứu đánh giá độ phức tạp của thuật toán, độ hiệu quả của thuật toán người ta xét độ đo về khía cạnh ước lượng toán học chứ không xét theo góc độ một chương trình dùng một ngôn ngữ cụ thể cài đặt thuật toán đó. Chương trình chỉ dùng để xác minh cho độ phức tạp đã tính toán ra mà thôi.

Một thuật toán có thể được lập trình bằng nhiều ngôn ngữ khác nhau như Pascal hay C++, C#, Java..., có thể chạy trên máy 32 bit hay 64 bit... nên thời gian thực hiện chương trình có thể khác nhau. Vì vậy các nhà khoa học máy tính tiền bối đã đưa ra khái niệm hàm O- lớn để đánh giá khái quát thời gian thực hiện một thuật toán, có tính chất ước lượng chứ không phải hoàn toàn chính xác, độ phức tạp mang tính tiệm cận (cận trên hay cận dưới).

Như trong ví dụ thuật toán trên, dù được lập trình bằng Pascal hay C++ hay bằng ngôn ngữ gì nữa thì thời gian thực hiện nó cũng chỉ có thể là $3n, 4n$ hay không quá cn với c là một hằng nguyên bất kỳ.

Vì vậy, độ phức tạp của thuật toán được định nghĩa là độ lớn của hàm $T(n)$, được gọi bằng hàm O lớn của hàm $f(n) : T(n)=O(f(n))$, nếu và chỉ nếu tồn tại một hằng số $c>0$ và một số $n_0>0$ sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$; trong đó $T(n)$ và $f(n)$ là các hàm thực không âm.

Trong lý thuyết phân tích thuật toán, người ta đã đưa ra một số cấp thời gian chuẩn thực hiện thuật toán như sau :

$O(1)$ - Độ phức tạp hằng số, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ không phụ thuộc đầu vào n , ví dụ : các thao tác hệ thống, hay mỗi phép $+, -, *, /, \%, \&\&, ||, !$

$O(\log(n))$ - Độ phức tạp logarit. Ví dụ thuật toán Euclid tìm USCLN, với $n = 8$ thì $\log(8)=3$.

$O(n)$ - Độ phức tạp tuyến tính, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ phụ thuộc đầu vào n .

$O(n\log n)$ - Độ phức tạp $n\log n$ (với logarit thập phân $\log(n)$ thì $O(n\log n) < O(n)$)

Chú ý là độ phức tạp logarit $O(\log n) < O(n\log n) < O(n)$, xem :

	LOG	n	LOG(n)
logarit nhị phân	lg(n)	8	3
logarit tự liên -e	ln(n)	8	2.079441542
logarit thập phân	log(n)	8	0.903089987

$O(P(n))$ - Độ phức tạp đa thức, P là đa thức bậc từ 2 trở lên. Ví dụ bài toán tính định thức ma trận.

$O(n^2)$ - Độ phức tạp bình phương

$O(2^n)$ - Độ phức tạp số mũ (2^n)

Với các chỉ tiêu trên đây, ta đã có một độ đo để đánh giá, so sánh thuật toán nào tốt hơn thuật toán nào khi giải quyết 1 bài toán..

2.3.Các quy tắc đánh giá thời gian thực hiện thuật toán

Quy tắc tổng : nếu $T_1(n) = O(f_1(n))$ và $T_2(n) = O(f_2(n))$

thì $T_1(n) + T_2(n) = O(\max(f_1(n), f_2(n)))$

Thật vậy :

vì $T_1(n) = O(f_1(n))$ nên $\exists c_1, n_1$ sao cho $T_1(n) \leq c_1 f_1(n)$ với $\forall n \geq n_1$, ($\exists$ đọc là *tồn tại*)

$T_2(n) = O(f_2(n))$ nên $\exists c_2, n_2$ sao cho $T_2(n) \leq c_2 f_2(n)$ với $\forall n \geq n_2$, ($\forall$ đọc là *với mọi*)

Đặt $n_0 = \max(n_1, n_2)$, khi đó với $\forall n \geq n_0$ ta có :

$$\begin{aligned} T_1(n) + T_2(n) &\leq (c_1 + c_2) \max(f_1(n), f_2(n)) = c \max(f_1(n), f_2(n)) \text{ với } c = c_1 + c_2 \\ &= O(\max(f_1(n), f_2(n))) \end{aligned}$$

Tổng quát : khi một bài toán đ-ợc chia thành nhiều bài toán con thì **Độ phức tạp** thuật toán của bài toán sẽ lấy theo cấp **Độ phức tạp lớn nhất** trong các cấp độ thành phần.

2.4.Đánh giá thời gian thực hiện các câu lệnh C,C++ và Pascal.

Để đánh giá độ phức tạp thực hiện một chương trình, ta phải biết đ-ợc từng câu lệnh thực hiện bao nhiêu lần.

Tùy theo loại lệnh, ta có các chỉ tiêu đánh giá nh- sau:

1.Lệnh gán(=, :=), lệnh nhập dữ liệu (cin>>, read) , lệnh hiện dữ liệu(cout<<, print, write) có thời gian thực hiện là $O(1)$;

2.Lệnh phức hợp để trong cặp {...}, begin ... end đ-ợc đánh giá theo quy tắc tổng ở trên ;

3.Lệnh if <điều kiện> s1 else s2 ;

IF <điều kiện> THEN S1 ELSE S2;

Độ phức tạp là $O(\max(f_1(n), f_2(n)))$, trong đó $f_1(n), f_2(n)$ là cận trên của độ phức tạp S1, S2.

4.Lệnh witch case, CASE đ-ợc đánh giá nh- lệnh if.

5.Lệnh lặp for,while, do...while,repeat ...until, while..do... : nếu độ phức tạp thực hiện thân vòng lặp là $f(n)$ và số lần lặp tối đa là $g(n)$ thì độ phức tạp thực hiện một lệnh lặp là $O(f(n)*g(n))$.

6.Đối với hàm đệ quy.

Xét hàm đệ quy tính giai thừa sau.

```
int giaithua(int n)
{ if (n <= 1)
  return 1;
  return n*giaithua(n-1);
}
```

Ta đặt độ phức tạp thực hiện hàm với n là $T(n)$. Với $n=1$ thì độ phức tạp thực hiện hàm là $O(1)$. Với $n > 1$, độ phức tạp thực hiện hàm chính là thời gian thực hiện lệnh gán $giaithua = n*giaithua(n-1)$ và $return\ giaithua$, tức là bằng $O(1)$ cộng với $T(n-1)$.

Ta có quan hệ đệ quy :

$$T(1) = O(1);$$

$$T(n) = O(1) + T(n-1)$$

Ta bắt đầu từ $T(n)$ cho đến $T(n-1), T(n-2), \dots, T(2), T(1)$:

$$T(n) = O(1) + O(1) + \dots + O(1) + T(1) = (n-1)O(1) + T(1) = nc_1 = O(n).$$

2.5. Phân tích một số thuật toán

2.5.1. Hàm fibonacci

```
int fibonacci(int n);
{ if (n < 2)
  return 1;
  return fibonacci(n-1) + fibonacci(n-2);
}
```

			1					
				1		2		
		1		2		3		
	1		2		3		5	
1		2		3		5		8
.....								

Ta có quan hệ : Với $n < 2$ và 1 hằng số c , $T(n) = c$

 Với $n \geq 2$, $T(n) = T(n-1) + T(n-2)$

Ng- ời ta đã tính được thời gian thực hiện hàm fibonacci có độ phức tạp $T(n) = O(\phi^n)$,

với $\phi = (1 + \sqrt{5})/2$. Thời gian thực hiện này là hàm mũ, vì vậy với n đủ lớn thì hàm này không hiệu quả.

2.5.2. Thuật toán Euclid - tìm ước số chung lớn nhất

```
#include <iostream>
using namespace std;
int uscln(int m, int n)
{ int r ;
  r =m % n;
  while ( r != 0){ m = n; n = r; r = m % n ; }
  return n;
}
int main()
{ cout << "\n\n  THUAT TOAN EUCLID TIM USCLN " << endl;
  int A,B;
  cout<< "\n Nhap A,B " ; cin>>A>>B;
  cout<<" \n USCLN cua "<<A<<" va "<<B<<" la : ";
  cout << uscln(A, B)<<" \n\n";
  system("pause");
  return 0;
}
```

Xét thời gian thực hiện trong thân thuật toán : **while (r != 0){ m = n; n = r; r = m % n ; }**

$$m = n.q_1 + r_1, \quad 0 \leq r_1 < n$$

$$n = r_1.q_2 + r_2, \quad 0 \leq r_2 < r_1$$

Nếu $r_1 > n/2$ thì suy ra $q_2 = 1$ (vì nếu $q_2 \geq 2$ thì $(số > n/2) * q_2 > n$ là không thể được)
, tức là $n = r_1 + r_2, \Rightarrow r_2 < n/2$

Còn $r_1 < n/2$ thì cũng suy ra $r_2 < n/2$ (vì $r_2 = n \% r_1$, phần dư phải nhỏ hơn số chia).

Nghĩa là trong mọi trường hợp, $r_2 < n/2$.

Ví dụ : $m=20, n=12$;

Lần thực hiện thứ nhất : $m=20 ; n=12, r_1 = m \% n = 8$; vì $(r_1 \neq 0)$ nên lặp while

Lần thực hiện thứ hai : $m=12 ; n=8, r_2 = m \% n = 4$; vì $(r_2 \neq 0)$ nên while

....

Nh- vậy, cứ sau hai lần thực hiện thân vòng lặp thì số d- sẽ nhỏ hơn một nửa của n.

Giả sử k là số nguyên lớn nhất sao cho $2^k \leq n$. Từ bất đẳng thức $2^k \leq n$ ta có $k \leq \log_2 n$.

(xem lại cách tính logarit của 1 số : $n=2^k \Rightarrow k = \log_2 n$)

Kết quả , độ phức tạp thuật toán sẽ là $O(\log_2 n)$.

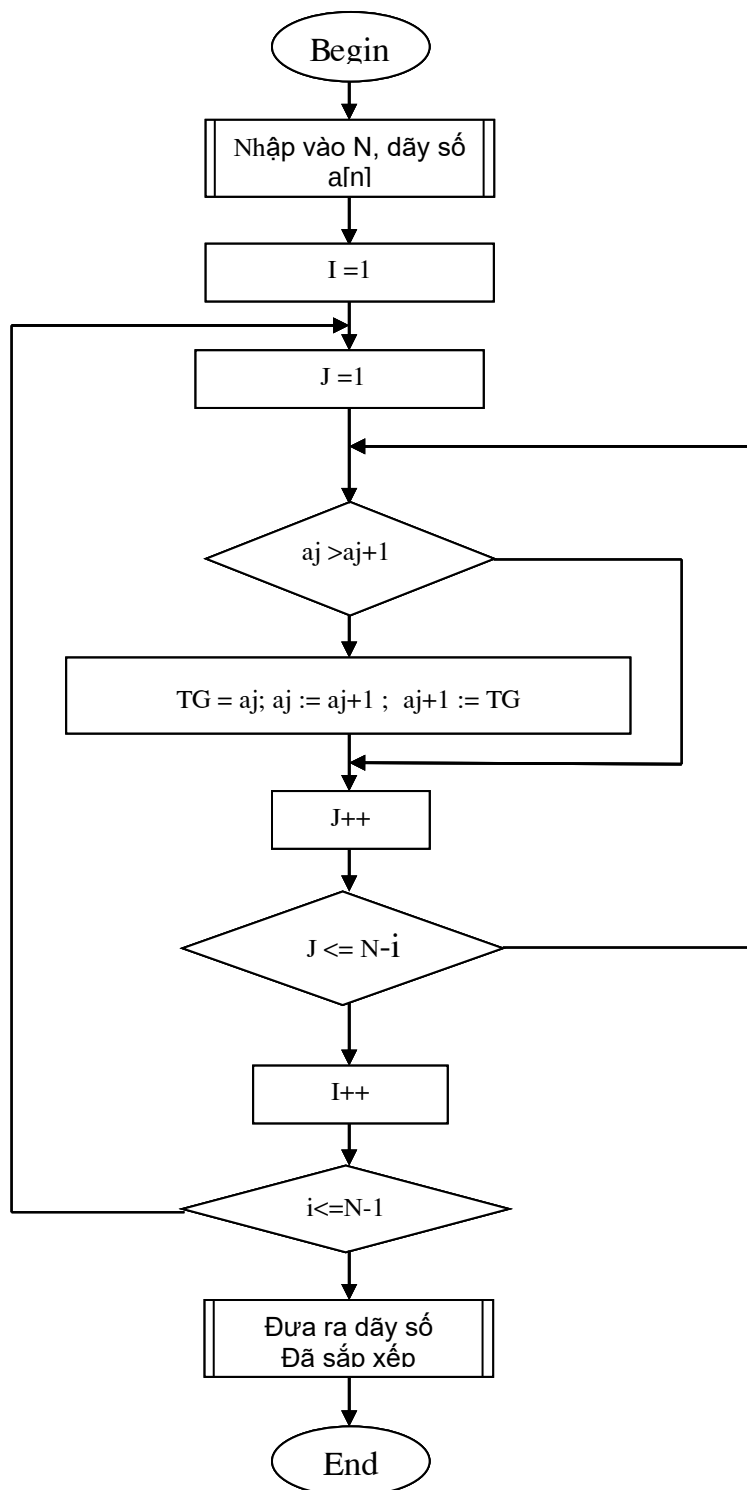
2.5.3. Độ phức tạp thuật toán sắp xếp nổi bọt

Chương trình:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{ cout << "\n\n  SAP XEP NOI BOT  " << endl;
  int i,j,n;
  float tg;
  float a[30];
  cout<<"\n\n  Nhap n :";cin>>n;
  for(i=1;i<=n;i++)
  { cout<<"\n  Nhap a["<<i<<"]= ";cin>>a[i];
  }
  for(i=1;i<=n-1;i++)
  for(j=1;j<=n - i;j++)
    { if(a[j]>a[j+1]){tg=a[j];a[j]=a[j+1];a[j+1]=tg;}
    }
  cout<<"\n\n  Day da sap xep : \n\n";
  cout<<setw(20);
  for(i=1;i<=n;i++) cout<<a[i]<<" , ";
  cout<<"\n\n";
  system("pause");
  return 0;
}
```

Sơ đồ thuật toán :



Theo sơ đồ, ta chỉ cần đếm số vòng lặp theo i và j .

Vòng lặp ngoài chạy theo i từ 1 đến $(n-1)$

Với mỗi i , vòng lặp trong j sẽ chạy từ 1 đến $(n-i)$,

$i = 1$, vòng lặp j trong chạy $n-1$ lần

$i = 2$, vòng lặp j trong chạy $n-2$ lần

.....

$i = n-1$, vòng lặp j trong chạy $n-(n-1) = 1$ lần

như vậy tổng số lần lặp chính là tổng của n số tự nhiên đầu tiên

$$1+2+3+\dots+(n-2)+(n-1)+n$$

, có công thức là :

$$n(n-1)/2 = n^2/2 - n/2$$

Cộng thêm n lần lặp nhập dữ liệu và n lần hiển dữ liệu cùng với một số thao tác hệ thống, bỏ qua các hằng số, độ phức tạp của thuật toán sắp xếp nổi bọt sẽ là $O(n^2)$.

Với độ phức tạp này cho ta thấy, khi n đủ lớn thì thuật toán sắp xếp nổi bọt là không hiệu quả.

2.6. Một số ký hiệu độ phức tạp khác

Ngoài ký hiệu O-lớn, người ta còn dùng thêm một số ký hiệu độ phức tạp khác, tùy theo mức độ đánh giá độ phức tạp, dựa theo từng định nghĩa.

2.6.1. o nhỏ- o(.)

Định nghĩa 1: $T(n)=o(f(n))$ nếu với mọi hằng số c, tồn tại hằng số $n_0 > 0$ sao cho $T(n) < cf(n)$ với mọi $n \geq n_0$.

Như vậy, $o(.)$ là **bằng cận trên** của thời gian thực hiện thuật toán.

Ngoài ra, còn định nghĩa khác cho $o(.)$:

Định nghĩa 2: $T(n)=o(f(n))$ nếu
$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$$

2.6.2. Độ phức tạp Omega - $\Omega(n)$

Định nghĩa : $T(n)=\Omega(f(n))$ nếu tồn tại hằng số c, $n_0 > 0$ sao cho $T(n) \geq cf(n)$ với mọi $n \geq n_0$.

Như vậy, $\Omega(n)$ - độ phức tạp Omega là **cận dưới** của thời gian thực hiện thuật toán.

2.6.3. Độ phức tạp Teta- $\Theta(.)$

Định nghĩa : $T(n) = \Theta(f(n))$ nếu $T(n) = O(f(n))$ và $T(n) = \Omega(n)$.

Như vậy, độ phức tạp **Teta- $\Theta(.)$** là khi cận trên **$O(.)$** và cận dưới **$\Omega(.)$** trùng với nhau.

Bài tập

1. Tính độ phức tạp của thuật toán sắp xếp thường, so sánh với độ phức tạp của thuật toán sắp xếp nổi bọt.

2. Tính độ phức tạp của thuật toán tìm kiếm theo phương pháp chia đôi (nhị phân).